

# Competitive Programming Notebook

py.Sandu

## Contents

|          |                                          |          |
|----------|------------------------------------------|----------|
| <b>1</b> | <b>Graph</b>                             | <b>2</b> |
| 1.1      | Reverse Graph . . . . .                  | 2        |
| 1.2      | Eulerean Path . . . . .                  | 2        |
| 1.3      | Ford-fulkerson . . . . .                 | 2        |
| 1.4      | Mst Prim . . . . .                       | 2        |
| 1.5      | Dijkstra . . . . .                       | 3        |
| 1.6      | Floyd-warshall . . . . .                 | 3        |
| 1.7      | Mst Kruskal . . . . .                    | 3        |
| 1.8      | Bellman-ford . . . . .                   | 3        |
| 1.9      | Bfs . . . . .                            | 4        |
| 1.10     | Dfs . . . . .                            | 4        |
| <b>2</b> | <b>Geometry</b>                          | <b>4</b> |
| 2.1      | Winding Number . . . . .                 | 4        |
| 2.2      | Isleft . . . . .                         | 4        |
| 2.3      | Point . . . . .                          | 4        |
| <b>3</b> | <b>String</b>                            | <b>4</b> |
| 3.1      | Kmp . . . . .                            | 4        |
| <b>4</b> | <b>General</b>                           | <b>5</b> |
| <b>5</b> | <b>DP</b>                                | <b>5</b> |
| 5.1      | Coins Change . . . . .                   | 5        |
| 5.2      | Kadane Algorithm . . . . .               | 5        |
| 5.3      | Knapsack . . . . .                       | 5        |
| 5.4      | Longest Increasing Subsequence . . . . . | 5        |
| <b>6</b> | <b>Math</b>                              | <b>6</b> |
| 6.1      | Matrix Multiplication . . . . .          | 6        |
| 6.2      | Modular Exponentiation . . . . .         | 6        |
| 6.3      | Solve Linear System . . . . .            | 6        |
| 6.4      | Matrix Fibonacci . . . . .               | 6        |
| 6.5      | Siege Of Eratosthenes . . . . .          | 6        |
| 6.6      | Extended Euclidean Algorithm . . . . .   | 7        |
| <b>7</b> | <b>DS</b>                                | <b>7</b> |
| 7.1      | Monotonic Stack . . . . .                | 7        |
| 7.2      | Dsu . . . . .                            | 7        |
| 7.3      | Sparsetable . . . . .                    | 8        |

# 1 Graph

## 1.1 Reverse Graph

```

1 // Reverse the edges of a directed graph
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V + E)$ 
4
5 Graph reverseGraph(const Graph& graph) {
6     int V = graph.size();
7     Graph reversedGraph(V);
8
9     for (int u = 0; u < V; u++) {
10         for (int v : graph[u]) {
11             reversedGraph[v].push_back(u);
12         }
13     }
14
15     return reversedGraph;
16 }

```

## 1.2 Eulerean Path

```

1 // Find the Eulerian path in a graph using Hierholzer
  's algorithm
2 // Time complexity:  $O(E)$  where E is the number of
  edges in the graph
3
4 void findEulerianPath(const vector<vector<int>>&
  graph, int start, vector<int>& path) {
5     stack<int> s;
6     s.push(start);
7
8     while (!s.empty()) {
9         int v = s.top();
10        if (graph[v].empty()) {
11            path.push_back(v);
12            s.pop();
13        } else {
14            int u = graph[v].back();
15            graph[v].pop_back(); // Remove the edge
16            from v to u
17            s.push(u);
18        }
19    }

```

## 1.3 Ford-fulkerson

```

1 // Find maximum flow in graph
2 // Time complexity:  $O(|V| * E^2)$ 
3 // Space complexity:  $O(V)$ 
4 bool bfs(int residual[V][V], int start, int target,
  vi parent){
5     vector<bool> visited(V, false);
6
7     queue<int> q;
8     q.push(start);
9     visited[start] = true;
10    parent[start] = -1;
11
12    while(!q.empty()){
13        int u = q.front();
14        q.pop();
15
16        for(int v = 0 ; v <= V ; v++){
17            if(!visited[v] and residual[u][v] > 0){
18                if(v == target){
19                    parent[v] = u;
20                    return true;
21                }

```

```

22                q.push(v);
23                parent[v] = u;
24                visited[v] = true;
25            }
26        }
27    }
28    return false;
29 }
30
31 int fordFulkerson(int g[V][V], int start, int target)
  {
32     int u, v;
33     int residual[V][V];
34
35     for(u = 0 ; u < V ; u++){
36         for(v = 0 ; v < V ; v++){
37             residual[u][v] = g[u][v];
38         }
39     }
40     int maxFlow = 0;
41
42     while(bfs(residual, start, target, parent)){
43         int pathFlow = INT_MAX;
44         for(v = target ; v != start ; v = parent[v]){
45             u = parent[v];
46             pathFlow = min(pathFlow, residual[u][v]);
47         }
48
49         for(v = target ; v != start ; v = parent[v]){
50             u = parent[v];
51             residual[u][v] -= pathFlow;
52             residual[v][u] += pathFlow;
53         }
54         maxFlow += pathFlow;
55     }
56     return maxFlow;
57 }

```

## 1.4 Mst Prim

```

1 // Prim's algorithm for finding the Minimum Spanning
  Tree
2 // Time complexity:  $O(E \log V)$  where E is the number
  of edges and V is the number of vertices
3
4 struct Edge {
5     int u, v, weight;
6     bool operator<(const Edge& other) const {
7         return weight > other.weight; // Min-heap
8     }
9 };
10
11 vector<Edge> primMST(const vector<vector<int>>& graph
  ) {
12     int n = graph.size();
13     vector<bool> visited(n, false);
14     vector<Edge> mst;
15     priority_queue<Edge> pq;
16
17     // Start with the first vertex
18     visited[0] = true;
19     for (int i = 0; i < n; ++i) {
20         if (graph[0][i] != 0) {
21             pq.push({0, i, graph[0][i]});
22         }
23     }
24
25     while (!pq.empty()) {
26         Edge current = pq.top();
27         pq.pop();
28
29         if (visited[current.v]) continue;
30     }

```

```

31     visited[current.v] = true;
32     mst.push_back(current);
33
34     for (int i = 0; i < n; ++i) {
35         if (!visited[i] && graph[current.v][i] !=
36             0) {
37             pq.push({current.v, i, graph[current.
38                 v][i]});
39         }
40     }
41     return mst;
42 }

```

## 1.5 Dijkstra

```

1 // Find shortest path from a source vertex to all
  // other vertices in a graph with non-negative edge
  // weights
2 // Time Complexity:  $O((V + E) \log V)$ 
3 // Space Complexity:  $O(V)$ 
4
5 void dijkstra(const Graph& graph, int src) {
6     int V = graph.size();
7     vector<int> dist(V, INT_MAX);
8     dist[src] = 0;
9
10    priority_queue<pair<int, int>, vector<pair<int,
11        int>>, greater<pair<int, int>>> pq; // Min-heap
12    priority queue
13    pq.push({0, src});
14
15    while (!pq.empty()) {
16        int u = pq.top().second;
17        pq.pop();
18
19        for (const auto& adj : graph[u]) {
20            int v = adj.first;
21            int weight = adj.second;
22
23            if (dist[u] != INT_MAX && dist[u] +
24                weight < dist[v]) {
25                dist[v] = dist[u] + weight;
26                pq.push({dist[v], v});
27            }
28        }
29    }
30 }

```

## 1.6 Floyd-warshall

```

1 // Find the shortest paths in a weighted graph with
  // positive or negative edge weights (but with no
  // negative cycles).
2 // Time Complexity:  $O(V^3)$ 
3 // Space Complexity:  $O(V^2)$ 
4
5 void floydWarshall(Graph& graph, vector<vector<int>>&
6     dist) {
7     int V = graph.size();
8
9     for(int i = 0; i < V; i++){
10        for (int j = 0; j < V; j++) {
11            if (i == j) {
12                dist[i][j] = 0;
13            } else if (dist[i][j] == 0) {
14                dist[i][j] = INT_MAX; // No edge
15            }
16        }
17    }

```

```

18     for (int k = 0; k < V; k++) {
19         for (int i = 0; i < V; i++) {
20             for (int j = 0; j < V; j++) {
21                 if (dist[i][k] != INT_MAX && dist[k][
22                     j] != INT_MAX) {
23                     dist[i][j] = min(dist[i][j], dist
24                         [i][k] + dist[k][j]);
25                 }
26             }
27         }
28     }
29 }

```

## 1.7 Mst Kruskal

```

1 // Kruskal algorithm for finding the Minimum Spanning
  // Tree
2 // Time Complexity:  $O(E \log E)$ 
3 // Space complexity:  $O(V + E)$ 
4 // Use ds/dsu.cpp template
5
6 // edges: {weight, u, v}
7 vector<vector<int>> kruskalMST(vector<array<int, 3>>&
8     edges, int n){
9     int m = (int)edges.size();
10    sort(edges.begin(), edges.end());
11    vector<vector<int>> mst(n+1);
12    DSU dsu(n+1);
13    for(int i=0; i<m; i++){
14        int a = edges[i][1], b = edges[i][2];
15        if(dsu.merge(a, b)){
16            mst[a].push_back(b);
17            mst[b].push_back(a);
18        }
19    }
20    return mst;
21 }

```

## 1.8 Bellman-ford

```

1 // Find the shortest path from a source vertex to all
  // other vertices in a graph with negative weight
  // edges and detect negative weight cycles.
2 // Time Complexity:  $O(V * E)$ 
3 // Space Complexity:  $O(V)$ 
4
5 void bellmanFord(Graph& graph, int src) {
6     int V = graph.size();
7     vector<int> dist(V, INT_MAX);
8     dist[src] = 0;
9
10    for (int i = 1; i < V; i++) {
11        for (int u = 0; u < V; u++) {
12            for (int v = 0; v < V; v++) {
13                if (graph[u][v] != 0 && dist[u] !=
14                    INT_MAX && dist[u] + graph[u][v] < dist[v]) {
15                    dist[v] = dist[u] + graph[u][v];
16                }
17            }
18        }
19    }
20
21    // Check for negative weight cycles
22    for (int u = 0; u < V; u++) {
23        for (int v = 0; v < V; v++) {
24            if (graph[u][v] != 0 && dist[u] !=
25                INT_MAX && dist[u] + graph[u][v] < dist[v]) {
26                cout << "Graph contains negative
27                    weight cycle" << endl;
28                return;
29            }
30        }
31    }
32 }

```

```
28     }
29 }
```

## 1.9 Bfs

```
1 // Breadth First Search (BFS)
2 // Time Complexity: O(V + E)
3 // Space Complexity: O(V)
4 void bfs(const Graph& graph, int start) {
5     int V = graph.size();
6     vector<bool> visited(V, false);
7     queue<int> q;
8
9     visited[start] = true;
10    q.push(start);
11
12    while (!q.empty()) {
13        int vertex = q.front();
14        q.pop();
15        cout << vertex << " ";
16
17        for (int neighbor : graph[vertex]) {
18            if (!visited[neighbor]) {
19                visited[neighbor] = true;
20                q.push(neighbor);
21            }
22        }
23    }
24 }
```

## 1.10 Dfs

```
1 // Depth First Search (DFS)
2 // Time Complexity: O(V + E)
3 // Space Complexity: O(V)
4
5 // Recursive DFS
6 void dfs(const Graph& graph, int vertex, vector<bool>
7     & visited) {
8     visited[vertex] = true;
9
10    for (int neighbor : graph[vertex]) {
11        if (!visited[neighbor]) {
12            dfs(graph, neighbor, visited);
13        }
14    }
15
16 // Iterative DFS
17 void dfsIterative(const Graph& graph, int start) {
18     int V = graph.size();
19     vector<bool> visited(V, false);
20     stack<int> s;
21     s.push(start);
22
23     while (!s.empty()) {
24         int vertex = s.top();
25         s.pop();
26
27         if (!visited[vertex]) {
28             visited[vertex] = true;
29
30             for (int neighbor : graph[vertex]) {
31                 if (!visited[neighbor]) {
32                     s.push(neighbor);
33                 }
34             }
35         }
36     }
```

## 2 Geometry

### 2.1 Winding Number

```
1 // Check if a point is inside a polygon using the
   winding number algorithm
2
3 bool isInsidePolygon(const Point& p, const vector<
4     Point>& polygon) {
5     int windingNumber = 0;
6     int n = polygon.size();
7
8     for (int i = 0; i < n; i++) {
9         const Point& v1 = polygon[i];
10        const Point& v2 = polygon[(i + 1) % n];
11
12        if (v1.y <= p.y) {
13            if (v2.y > p.y && isLeft(p, v1, v2)) {
14                ++windingNumber;
15            }
16        } else {
17            if (v2.y <= p.y && !isLeft(p, v1, v2)) {
18                --windingNumber;
19            }
20        }
21    }
22    return windingNumber != 0;
23 }
```

### 2.2 Isleft

```
1 // Check if a point is to the left of a segment
   defined by two points
2 bool isLeft(const Point& p, const Point& v1, const
3     Point& v2) {
4     return (v2.x - v1.x) * (p.y - v1.y) - (v2.y - v1.
5         y) * (p.x - v1.x) > 0;
6 }
```

### 2.3 Point

```
1 struct Point {
2     double x, y;
3 };
```

## 3 String

### 3.1 Kmp

```
1 // Find all occurrences of a pattern in a given text
   using the Knuth-Morris-Pratt (KMP) algorithm
2 // Time complexity: O(n + m) where n is the length of
   the text and m is the length of the pattern
3
4 vector<int> LPS(const string& pattern) {
5     int m = pattern.size();
6     vector<int> lps(m, 0);
7     int len = 0; // length of the previous longest
   prefix suffix
8     int i = 1;
9
10    while (i < m) {
11        if (pattern[i] == pattern[len]) {
12            len++;
13            lps[i] = len;
14            i++;
15        } else {
16            if (len != 0) {
17                len = lps[len - 1];
```

```

18         } else {
19             lps[i] = 0;
20             i++;
21         }
22     }
23 }
24 return lps;
25 }
26
27 vector<int> KMP(const string& text, const string&
    pattern) {
28     vector<int> lps = LPS(pattern);
29     vector<int> occurrences;
30     int n = text.size();
31     int m = pattern.size();
32     int i = 0; // index for text
33     int j = 0; // index for pattern
34
35     while (i < n) {
36         if (text[i] == pattern[j]) {
37             i++;
38             j++;
39         }
40         if (j == m) {
41             occurrences.push_back(i - j); // Pattern
42             found at index (i - j)
43             j = lps[j - 1]; // Continue to search for
44             the next match
45         } else if (i < n && text[i] != pattern[j]) {
46             if (j != 0) {
47                 j = lps[j - 1];
48             } else {
49                 i++;
50             }
51         }
52     }
53     return occurrences;
54 }

```

## 4 General

## 5 DP

### 5.1 Coins Change

```

1 // Find the minimum number of coins needed to make a
2 // given amount using a set of coin denominations
3 // Time complexity: O(n * m) where n is the amount
4 // and m is the number of coin denominations
5
6 int coinChange(const vector<int>& coins, int amount)
7 {
8     vector<int> dp(amount + 1, INT_MAX);
9     dp[0] = 0;
10
11     for (int i = 1; i <= amount; i++) {
12         for (const int& coin : coins) {
13             if (coin <= i && dp[i - coin] != INT_MAX)
14                 dp[i] = min(dp[i], dp[i - coin] + 1);
15         }
16     }
17
18     return dp[amount] == INT_MAX ? -1 : dp[amount];
19 }

```

### 5.2 Kadane Algorithm

```

1 // Find maximum subarray sum using Kadane's algorithm

```

```

2 // Time complexity: O(n) where n is the length of the
3 // input array
4
5 int maxSubArraySum(const vector<int>& arr) {
6     int maxSum = arr[0];
7     int maxLocal = arr[0];
8
9     for (int i = 0; i < arr.size(); i++) {
10         maxLocal = max(arr[i], maxLocal + arr[i]);
11         maxSum = max(maxSum, maxLocal);
12     }
13
14     return maxSum;
15 }

```

### 5.3 Knapsack

```

1 // General take/not take dp problem
2 // Time Complexity: aprox. O(n * m)
3 // Space Complexity: O(n * m) / O(m)
4
5 // Top-down
6
7 const int MAXN = 1e3, MAXW = 1e3;
8
9 int knapsack_top_down(vector<int>& v, vector<int>& w,
10     int x){
11     int n = (int)v.size();
12     vector<vector<int>>> dp(MAXN, vector<int>(MAXW,
13         -1));
14     auto rec = [&](auto self, int i, int m) -> int {
15         if(i >= n || m == 0) return 0;
16         if(dp[i][m] != -1) return dp[i][m];
17         int not_take = self(self, i+1, m);
18         int take = 0;
19         if(m >= w[i]){
20             take = v[i] + self(self, i+1, m - w[i]);
21         }
22         return dp[i][m] = max(take, not_take);
23     };
24
25     return rec(rec, 0, x);
26 }
27
28 // Bottom-up
29
30 int knapsack_bottom_up(vector<int>& v, vector<int>& w,
31     int x){
32     int n = (int)v.size();
33     vector<int> dp(x+1, 0);
34
35     for(int i=0; i < n; i++){
36         for(int m=x; m >= w[i]; m--){
37             dp[m] = max(dp[m], v[i] + dp[m - w[i]]);
38         }
39     }
40
41     return dp[x];
42 }

```

### 5.4 Longest Increasing Subsequence

```

1 // Find the length of the longest increasing
2 // subsequence in a given array
3 // Time complexity: O(n log n) where n is the length
4 // of the input array
5
6 int lis(vector<int>& arr) {
7     vector<int> dp;
8     for (int num : arr) {
9         auto it = lower_bound(dp.begin(), dp.end(),
10             num);

```

```

8         if (it == dp.end()) {
9             dp.push_back(num);
10        } else {
11            *it = num;
12        }
13    }
14    return dp.size();
15 }

```

## 6 Math

### 6.1 Matrix Multiplication

```

1 // Multiply two matrices A and B, and return the
  // result in matrix C.
2 // Time Complexity: O(n^3)
3 // Space Complexity: O(n^2)
4
5 vector<vector<int>>> matrixMultiplication(const vector
  <vector<int>>>& A, const vector<vector<int>>>& B) {
6     int n = A.size();
7     vector<vector<int>>> C(n, vector<int>(n, 0)); //
  Initialize result matrix with zeros
8
9     for (int i = 0; i < n; i++) {
10        for (int j = 0; j < n; j++) {
11            for (int k = 0; k < n; k++) {
12                C[i][j] += A[i][k] * B[k][j];
13            }
14        }
15    }
16    return C;
17 }

```

### 6.2 Modular Exponentiation

```

1 // Calculates (x^n) % MOD
2 // Time Complexity: O(log n)
3 // Space Complexity: O(1)
4
5 int powMod(int x, int b, int MOD){
6     int res = 1;
7
8     while(b > 0){
9         if(b & 1) res = (res * x) % MOD;
10        x = (x * x) % MOD;
11        b >>= 1;
12    }
13    return res;
14 }

```

### 6.3 Solve Linear System

```

1 // Solve a system of linear equations using Cramer's
  // rule
2 // Time complexity: O(n^4) in the worst case
3
4 // Function to calculate the determinant of a matrix
5 double determinant(const vector<vector<double>>&
  matrix) {
6     int n = matrix.size();
7     if (n == 1) {
8         return matrix[0][0];
9     }
10    double det = 0.0;
11    for (int col = 0; col < n; col++) {
12        vector<vector<double>>> submatrix(n - 1,
  vector<double>(n - 1));
13        for (int i = 1; i < n; ++i) {
14            for (int j = 0; j < n; ++j) {
15                if (j < col) {

```

```

16                submatrix[i - 1][j] = matrix[i][j]
17            };
18            } else if (j > col) {
19                submatrix[i - 1][j - 1] = matrix[
20                i][j];
21            }
22            det += (col % 2 == 0 ? 1 : -1) * matrix[0][
23            col] * determinant(submatrix);
24        }
25    }
26
27 // Function to solve a system of linear equations
  // using Cramer's rule
28 vector<double> solveLinearSystem(const vector<vector<
  double>>& A, const vector<double>& b) // A is the
  // coefficient matrix and b is the constant vector
29 {
30     int n = A.size();
31
32     double detA = determinant(A);
33
34     vector<double> solution(n);
35     for (int i = 0; i < n; i++) {
36         vector<vector<double>>> Ai = A;
37         for (int j = 0; j < n; ++j) {
38             Ai[j][i] = b[j];
39         }
40         solution[i] = determinant(Ai) / detA; //
  Cramer's rule
41     }
42     return solution;
43 }

```

### 6.4 Matrix Fibonacci

```

1 // Calculate the Nth Fibonacci number using matrix
  // exponentiation.
2 // Time Complexity: O(log N)
3 // Space Complexity: O(1)
4
5 void matrixExponentiation(vector<vector<long long>>&
  matrix, int power) {
6     int n = matrix.size();
7     vector<vector<long long>>> result(n, vector<long
  long>(n, 0));
8
9     // Initialize result as the identity matrix
10    for (int i = 0; i < n; i++) {
11        result[i][i] = 1;
12    }
13
14    while (power) {
15        if (power % 2 == 1) {
16            matrixMultiplication(result, matrix,
  result);
17        }
18        matrixMultiplication(matrix, matrix, matrix);
19        power /= 2;
20    }
21
22    matrix = result;
23 }

```

### 6.5 Siege Of Eratosthenes

```

1 // Find all prime numbers up to a given limit using
  // the Sieve of Eratosthenes algorithm
2 // Time complexity: O(n log log n) where n is the
  // limit

```

```

3 // This idea can also be used to construct a table
  for the smallest prime factors,  $O(\log N)$  per
  query
4
5 vector<int> findPrimes(int limit) {
6     vector<bool> isPrime(limit + 1, true);
7     isPrime[0] = isPrime[1] = false;
8
9     for (int i = 2; i * i <= limit; i++) {
10         if (isPrime[i]) {
11             for (int j = i * i; j <= limit; j += i) {
12                 isPrime[j] = false;
13             }
14         }
15     }
16
17     vector<int> primes;
18     for (int i = 2; i <= limit; i++) {
19         if (isPrime[i]) {
20             primes.push_back(i);
21         }
22     }
23     return primes;
24 }
25
26 vector<int> buildSPF(int limit){
27     vector<int> spf(limit+1);
28     iota(spf.begin(), spf.end(), 0);
29     for(int i = 2; i * i <= limit; i++) {
30         if(spf[i] == i){
31             for(int j = i * i; j <= limit; j += i) {
32                 if(spf[j] == j) spf[j] = i;
33             }
34         }
35     }
36
37     return spf;
38 }
39
40 vector<int> getFactors(vector<int>& spf, int x){
41     vector<int> factors;
42     while(x > 1){
43         factors.push_back(spf[x]);
44         x /= spf[x];
45     }
46
47     return factors;
48 }

```

## 6.6 Extended Euclidean Algorithm

```

1 // Extended Euclidean Algorithm
2 // Returns (gcd(a, b), x, y) such that  $a*x + b*y =$ 
  gcd(a, b)
3
4 int extendedEuclidean(int a, int b, int &x, int &y){
5     if(a == 0){
6         x = 0;
7         y = 1;
8         return b;
9     }
10
11     int x1, y1;
12     int gcd = extendedEuclidean(b % a, a, x1, y1);
13     x = y1;
14     y = x1 - (b / a) * y1;
15     return gcd;
16 }

```

## 7 DS

### 7.1 Monotonic Stack

```

1 // Increasing Monotonic Stack
2 // Time Complexity:  $O(n)$ 
3 // Space Complexity:  $O(n)$ 
4
5 vector<int> nextGreaterElements(vector<int>& arr) {
6     int n = arr.size();
7     vector<int> res(n, -1);
8     stack<int> st;
9
10    for (int i = 0; i < 2 * n; i++) {
11        int num = arr[i % n];
12        while (!st.empty() && arr[st.top()] < num) {
13            res[st.top()] = num;
14            st.pop();
15        }
16        if (i < n) {
17            st.push(i);
18        }
19    }
20
21    return res;
22 }
23
24 // Decreasing Monotonic Stack
25 // Time Complexity:  $O(n)$ 
26 // Space Complexity:  $O(n)$ 
27
28 vector<int> nextSmallerElements(vector<int>& arr) {
29     int n = arr.size();
30     vector<int> res(n, -1);
31     stack<int> st;
32
33    for (int i = 0; i < 2 * n; ++i) {
34        int num = arr[i % n];
35        while (!st.empty() && arr[st.top()] > num) {
36            res[st.top()] = num;
37            st.pop();
38        }
39        if (i < n) {
40            st.push(i);
41        }
42    }
43
44    return res;
45 }

```

### 7.2 Dsu

```

1 // Disjoint Set Union(DSU)
2 // Time Complexity: approx.  $O(1)$ 
3 // Space Complexity:  $O(n)$ 
4
5 struct DSU{
6
7     vector<int> parent, size;
8
9     DSU(int n){
10         parent.assign(n+1, 0);
11         size.assign(n+1, 1);
12         iota(parent.begin(), parent.end(), 0);
13     }
14
15     int find(int v){
16         if(v == parent[v]) return v;
17         return parent[v] = find(parent[v]);
18     }
19
20     bool same(int a, int b){

```

```

21     return find(a) == find(b);
22 }
23
24 bool merge(int a, int b){
25     a = find(a);
26     b = find(b);
27     if(a != b){
28         if(size[a] < size[b]) swap(a, b);
29         parent[b] = a;
30         size[a] += size[b];
31         return true;
32     }
33     return false;
34 }
35
36 };

```

### 7.3 Sparsetable

```

1 // Sparse Table
2 // Time Complexity: O(n * log n) build, O(1) query
3 // Space complexity: O(n * log n)
4 // Idempotent operation f(a, a) = a
5
6 struct SparseTable{
7
8     int n, p;

```

```

9     vector<vector<int>> st;
10
11 SparseTable(vector<int> &arr){
12     n = (int) arr.size();
13     p = __lg(n);
14     st.assign(p+1, vector<int>(n, 0));
15
16     for(int j=0; j<n; j++) st[0][j] = arr[j];
17     for(int i = 1; i <= p; i++){
18         int k = (1ll << (i-1));
19         int lim = n - 2*k + 1;
20         for(int j=0; j<lim; j++){
21             st[i][j] = op(st[i-1][j], st[i-1][j+k
22         ]);
23     }
24 }
25
26 int queries(int l, int r){
27     int len = r - l + 1;
28     int p_local = __lg(len), k = (1ll << p_local)
29     ;
30     return op(st[p_local][l], st[p_local][r - k +
31     1]);
32 };

```